



School of Electrical and Computer Sciences
Indian Institute of Technology, Bhubaneswar

Light Communication between Decentralized Systems

Authors : Shishir Ravi Jois
Shrinivas B M
Supervised by : Dr. Srinivas Boppu
Affiliation : IIT Bhubaneswar
Branch : ECE
Date : 4th December 2025

Project Link: <https://github.com/Air-36/Light-Communication-between-Decentralized-Systems>

Demo Video: [Light communication using laser](#)

Table of Contents

1. Introduction	2
2. Circuit Diagram	3
3. Components Description	3
4. Software and Its Related Driving Concepts	7
5. Possible Improvements.....	9
6. Difficulties faced	10
7. Conclusion.....	10
8. References	10

1. Introduction

This project demonstrates a light-based communication mechanism between two independent embedded systems, where information is transmitted using a laser diode and received using a phototransistor. Each system operates autonomously, without shared wiring or a centralized controller, and coordination is achieved entirely through optical signalling. The transmitter converts keypad input into an 8-bit data frame, adds a small synchronization pattern, and sends the resulting sequence as timed light pulses generated through a hardware timer.

The receiver samples the incoming light at a fixed 5 kHz rate, detects the synchronization sequence through a sliding-window approach, and begins reconstructing each bit that follows. Once a complete byte is received, it is stored on an SD card and can be viewed on a TFT display. Additional controls allow clearing the stored file or displaying its content on demand.

The logic of the system, therefore integrates several key ideas: timer-driven bit timing, pattern-based synchronization, real-time optical sensing through analogue input, byte assembly, and persistent logging of the decoded data. This creates a reliable point-to-point optical link suitable for demonstrating how decentralized systems can communicate using light without relying on cables or radio communication.

2. Circuit Diagram

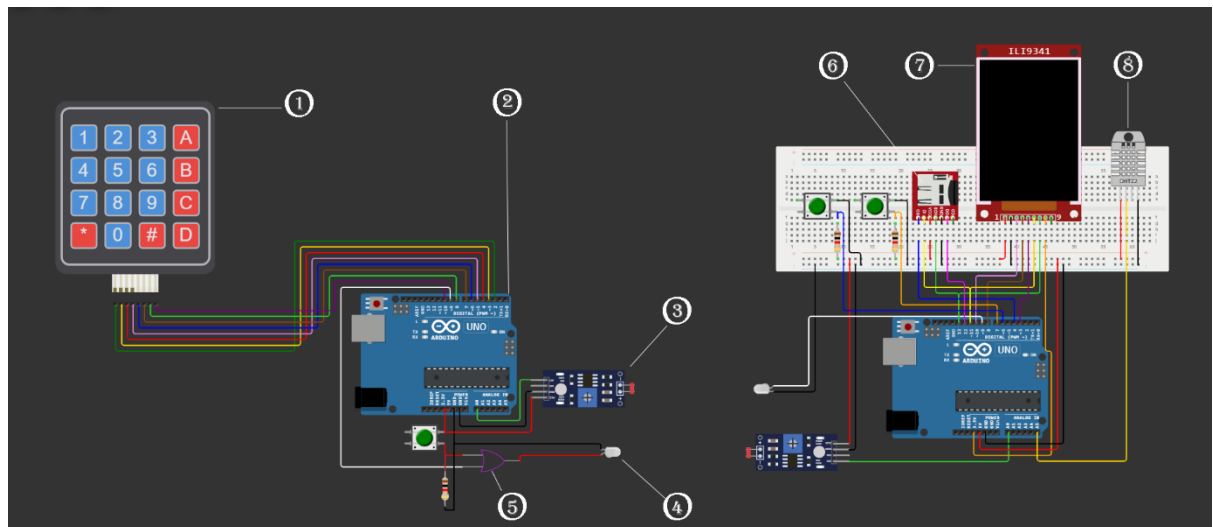


Fig. 2.a. Circuit Diagram.

3. Components Description

1. 4x4 Matrix Keypad

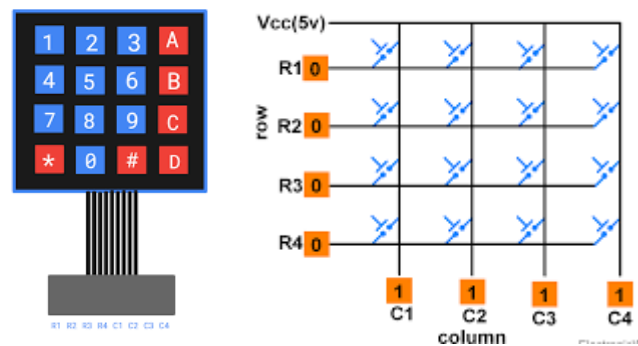
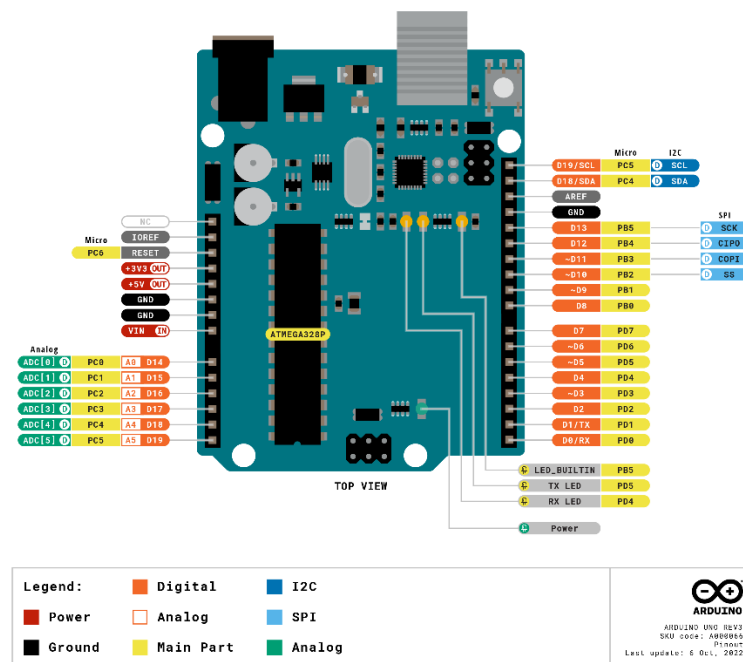


Fig. 3.a. Keypad Matrix and its functioning.

- Contains 16 keys arranged in rows and columns.
- Used for entering numbers or characters.
- Works by scanning row-column intersections.

2. Arduino UNO (Transmitter Node and Receiver Node)



- A semiconductor light sensor whose collector–emitter current increases with incident light.
- More sensitive and faster than LDRs.
- Converts light intensity into an electrical signal.
- Commonly used for light measurement, object detection, and optoelectronic switches.

4. Laser Diode Module (Transmitting device)

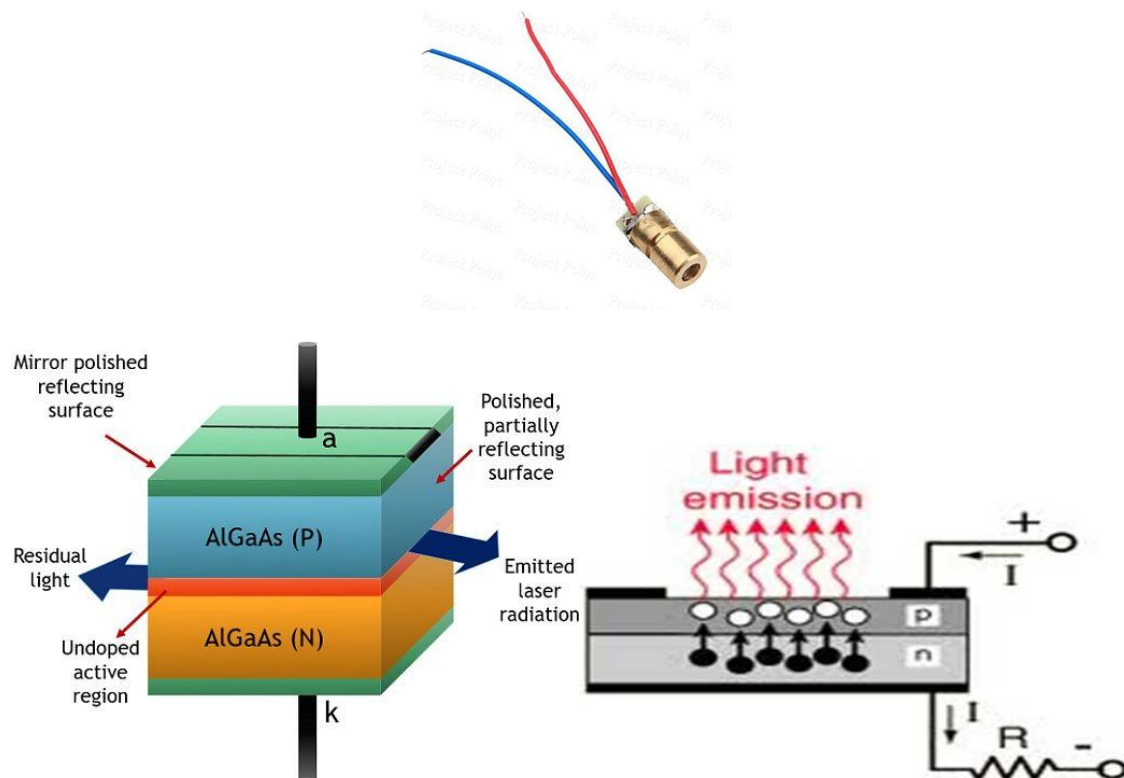


Fig. 3.d. Laser diode module and its functioning.

- Emits a narrow beam of focused light.
- Can be controlled using a digital signal.
- Used for alignment, sensing, or optical signalling.

5. OR – gate (IC SN7432N)

(OR gate IC in circuit diagram is illustrated by its logical symbol due to lack of module in CAD designs)

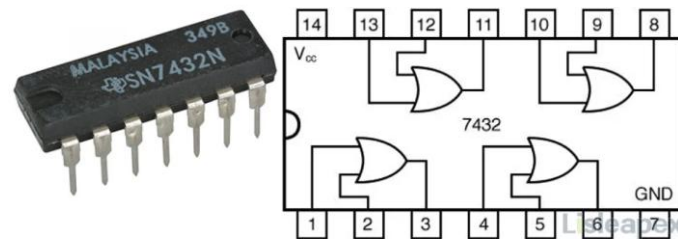


Fig. 3.e. OR gate IC and its internal connections.

- Contains four independent 2-input OR gates in a single IC package.
- Outputs HIGH (1) if any one or both inputs are HIGH.
- Operates typically on a 5V supply, compatible with TTL logic levels.
- Commonly used for decision-making circuits, alarm triggers, and logic control.

6. SD Card Module (Flash memory storage)



Fig. 3.f. SD card module.

- Interface for reading/writing SD memory cards.
- Communicates through SPI.
- Used for data logging or external storage.

7. TFT LCD Display (ILI9163C)

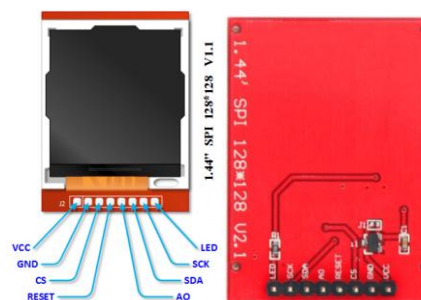


Fig. 3.g. TFT LCD display screen and connections.

- Small colour display supporting SPI communication.
- Can show text, images, and graphics.
- Used in portable devices and user interfaces.

8. HR-202 humidity sensor

(Circuit diagram illustrates DHT22 sensor due to lack of module in CAD designs)

***DHT11/22 preferred if available*

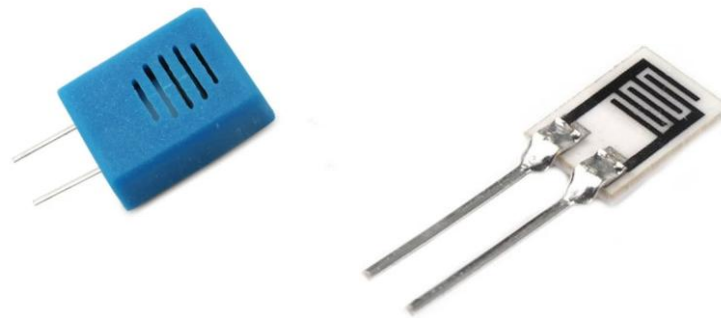


Fig. 3.h. HR-202 humidity sensor and its functioning.

- An analogue polymer humidity sensor whose resistance varies with environmental humidity.
- Requires an external circuit (like a voltage divider or signal conditioning) to convert resistance changes into voltage.
- Provides continuous humidity measurement, unlike DHT22, which gives processed digital output.

9. Other Basic Components

- **Switches:** to control certain conditional and manual operations.
- **Resistors:** acting as pull-up or pull-down in various passive devices, like pull-up in case of switches on display and SD card and also on humidity sensor, while pull down in case of noise generating switch or in conditioning phototransistor receivers.
- **Breadboard** and **wires** (Jumping cables).

4. Software and Its Related Driving Concepts

<< The transmitter and receiver code are available in the GitHub repository linked with this report and are not included here due to space constraints.>>

Click on the text below to read the respective code

[Transmitter node](#)

[Receiver node](#)

During the development process, the **Adafruit ST7735 and ST7789** display libraries were found to cause issues such as text misalignment and inconsistent rendering, mainly due to their internal buffer and coordinate handling. Although switching to the **Adafruit SSD1306** library would have provided a more stable and reliable text output, the replacement could not be completed because of time constraints. A similar situation occurred with the light-sensing component: the code was originally written for an LDR, but hardware testing showed that both the LDR and a solar cell had high switching times and inconsistent responses under rapid light changes. As a result, a photodiode was chosen for the final implementation due to its faster response and better accuracy; however, some references to the LDR remained in the code because the naming and library adjustments were not fully updated within the available timeframe.

During testing, it was observed that the LDR had a very slow switching response—approximately **1V per 100ms**—making it barely capable of supporting transmission rates in just the tens of hertz. The solar panel performed somewhat better, with a switching rate of about **1V per 10ms**, but its large surface area introduced a very low noise margin and made it highly sensitive to ambient light variations. In contrast, the phototransistor offered a much faster response of around **1V per 10μs** or less, providing clean and rapid transitions suitable for light-based communication. During these measurements, it was also found that the transmitting laser itself had a switching limitation of roughly **1V per few hundred microseconds**, which constrained the maximum practical transmission frequency to around 5 kHz. For these reasons, the phototransistor was ultimately selected for the final implementation, even though the code retained some references to the original LDR configuration.

Prescaler	CS12	CS11	CS10
No clock source (Timer/Counter stopped)	0	0	0
Clock / 1	0	0	1
Clock / 8	0	1	0
Clock / 64	0	1	1
Clock / 256	1	0	0
Clock / 1024	1	0	1

On the Arduino Uno, the ATmega328P's hardware timers allow precise frequency generation by configuring clock-division (prescaler) bits such as **CS10**, **CS11**, **CS12**, which determine how much the 16 MHz system clock is divided before feeding the timer. For example, setting **CS11 = 1** selects a prescaler of **8**, meaning the timer increments at $16 \text{ MHz} \div 8 = 2 \text{ MHz}$. Interrupts are managed through an **Interrupt Service Routine (ISR)**, which executes automatically when a timer event occurs. Global interrupt control instructions such as **sei()** (set enable interrupt) enable all interrupts, while instructions like **cli()** disable them when precise timing or critical operations are required.

To generate a specific frequency using **Timer1**, the value loaded into the **Output Compare Register A (OCR1A)** determines when the timer resets or toggles the output pin. The general formula is:

$$f = \frac{f_{\text{clock}}}{\text{prescaler} \times (\text{OCR1A} + 1)}$$

Here, the division by 2 occurs when the timer is configured in toggle mode (each compare event toggles the output, producing one full cycle every two toggles). Using a prescaler of 8 (CS11 = 1) and setting **OCR1A = 399**, the resulting frequency becomes:

$$f = \frac{16,000,000}{8 \times (399 + 1)} = \frac{16,000,000}{8 \times 400}$$

$$f = \frac{16,000,000}{3,200} = 5,000 \text{ Hz}$$

This produces a clean **5 kHz** output frequency. The timer counts from 0 to 399 at a 2 MHz tick rate, generating a compare match every 200 μs , and the pin toggles on each match, resulting in a 5 kHz square wave. Through this combination of prescaler selection, OCR1A calculation, and ISR-driven control, Arduino timers enable precise, hardware-based frequency generation and interrupt timing.

5. Possible Improvements

Several improvements can be made to enhance the performance and reliability of the current light-communication system. The data rate can be increased by replacing simple ON–OFF keying with more efficient schemes such as Manchester coding or PWM-based signalling, while using hardware timers on both the transmitter and receiver would provide far more precise bit timing compared to delay-based loops. The receiver side can be made significantly more robust by adding automatic gain control and incorporating a transimpedance amplifier to convert the phototransistor's current into a cleaner, higher-bandwidth voltage signal. Interrupt-driven edge detection would reduce timing jitter and improve decoding accuracy, and implementing parity or CRC checks would add error resilience to the link. On the hardware side, the laser's switching limitations could be improved by driving it through a dedicated transistor or MOSFET stage, enabling faster rise and fall times. Optical performance can be further enhanced by adding a band-pass filter matched to the laser wavelength to reduce ambient light interference, and by adding a fixed preamble pattern to each transmitted frame to simplify synchronization at the receiver. Finally, if higher speeds are desired, replacing the phototransistor with a reverse-biased photodiode and TIA could unlock much higher bandwidth, provided the transmitter is upgraded accordingly.

6. Difficulties faced

This project also required extensive testing and design iterations to achieve a “perfect” or highly reliable receiver. Developing such a receiver demanded careful tuning of the phototransistor circuitry, filtering methods, and threshold levels to ensure that the system could distinguish valid optical signals from noise under varying environmental conditions. Beyond this, several other constraints significantly shaped the development process. Cost limitations restricted the use of higher-quality lasers and precision driver circuits, which in turn limited the switching speed and reduced the attainable data rate. Similar budget constraints affected the microcontroller choice; working with an MCU that had limited memory, fewer hardware timers, no RTC functionality, and lower-resolution ADCs made precise timing and clean signal acquisition more challenging. Physical setup issues also posed difficulties, especially maintaining accurate alignment between the transmitter and receiver, as even minor misalignment led to signal loss or distortion. Expanding the system into a decentralized architecture added further complexity, because integrating multiple sensors and enabling two-way communication among several nodes required careful coordination to ensure that each device could function not just as a transmitter or receiver, but as an active participant in a distributed network. These combined challenges made the design, testing, and refinement of the system a demanding yet valuable learning process.

7. Conclusion

In conclusion, this project demonstrated the feasibility of using light as a medium for communication within a decentralized network of devices, despite several technical and practical constraints. Through iterative testing and careful hardware selection, a functional optical link was achieved using cost-effective components and a custom-designed receiver optimized for stability. Although limitations in component quality, microcontroller capability, and physical alignment restricted the maximum data rate and overall system scalability, the project successfully established two-way interaction among multiple nodes and highlighted clear paths for future improvement. Ultimately, the work provided valuable insight into optical communication principles, system integration, and the challenges of building a distributed network under real-world constraints.

8. References

<https://nerd-corner.com/arduino-timer-interrupts-how-to-program-arduino-registers/>

***Certain images used in this report were sourced or lightly edited from publicly available online materials solely for the purpose of illustration and simplifying explanations.*